

THAKUR COLLEGE OF ENGINEERING AND TECHNOLOGY

(Autonomous Institute, Affiliated to University of Mumbai)

RESEARCH FOUNDATION DOSSIER

Semester VII

Problem Intelligence, Research Design & Claim Formulation

Project Title: VendorHood-A Multitenant SaaS Online Retail System

Domain: Cloud Computing (Software as a Service - SaaS) & E-Commerce

Student Team: Aryan Goud (B.E COMP-A, RollNo.48), Gaurav Gautam (B.E COMP-A, RollNo.44), Nidhay Chavan (B.E COMP-A, RollNo.20)

Faculty Guide: Ms. Akshata Raut

1. Abstract

The rapid growth of e-commerce has created significant opportunities for businesses of all sizes. However, small and medium-sized enterprises (SMEs) often face substantial financial and technical barriers when adopting online retail platforms. Traditional e-commerce solutions generally require dedicated infrastructure, software licensing, customization, deployment, and continuous maintenance, resulting in high upfront investment and operational costs. These challenges discourage many small vendors from establishing an online presence.

This research proposes a Multi-Tenant Software-as-a-Service (SaaS) framework for online retail systems that enables multiple vendors to share a common application and infrastructure while maintaining logical isolation of their data and configurations. The proposed framework introduces a commission-based monetization model in which vendors pay only when successful transactions occur, thereby eliminating fixed subscription costs. The architecture incorporates shared-schema multi-tenancy, tenant-aware data isolation, template-driven storefront generation, secure payment split settlement, and domain-based tenant resolution.

The proposed framework significantly reduces deployment costs, improves resource utilization, simplifies storefront customization, and provides scalable infrastructure capable of supporting multiple tenants simultaneously. By combining architectural scalability with an affordable pricing model, the framework offers an effective solution for enabling small retailers to participate in digital commerce without substantial initial investment.

2. Problem Context and System Understanding

Problem Context

The rapid growth of digital commerce has created new opportunities for businesses to expand their market reach. However, small and medium-sized retailers often encounter significant financial and technical challenges when adopting online retail platforms. Traditional e-commerce solutions require substantial upfront investment in software development, infrastructure, licensing, customization, deployment, and ongoing maintenance. These costs create a major barrier for small businesses with limited capital and uncertain demand. In addition, maintaining separate applications and infrastructure for each retailer leads to resource duplication, increased operational expenses, and inefficient utilization of computing resources. As a result, many small vendors struggle to establish an online presence despite the growing demand for digital retail.

System Understanding

The proposed system is a **Multi-Tenant Software-as-a-Service (SaaS) framework** designed to provide a cost-effective and scalable online retail platform for multiple vendors. Instead of maintaining separate applications for each retailer, all tenants share a common application and infrastructure while their data remains logically isolated through tenant-aware mechanisms. The framework adopts a shared-schema database architecture, ensuring efficient resource utilization without compromising data security. It also incorporates a commission-based monetization model, where the platform earns revenue only when vendors successfully complete transactions, thereby eliminating fixed subscription costs. Storefront customization is achieved using a template-driven rendering engine that enables vendors to create personalized online stores without additional software development. Furthermore, the framework supports both subdomain-based and custom-domain routing for tenant identification and website hosting. While the platform manages tenant onboarding, storefront generation, payment processing, and tenant services, logistics and order fulfillment remain the responsibility of individual vendors. This architecture provides an affordable, scalable, and secure solution that enables small retailers to participate in digital commerce with minimal upfront investment.

3. Formal Problem Statement

Traditional e-commerce platforms require every online retailer to maintain independent application infrastructure, databases, software licenses, maintenance teams, and hosting resources. These requirements result in high capital expenditure, redundant resource utilization, and increased operational complexity, making digital transformation financially challenging for small and medium-sized businesses.

Although existing SaaS platforms partially address these issues through resource sharing, many lack integrated support for secure tenant isolation, flexible storefront customization, commission-based revenue models, and efficient domain management within a unified architecture.

Therefore, the problem addressed in this research is:

To design and develop a scalable, secure, and cost-effective Multi-Tenant SaaS framework for online retail systems that enables multiple vendors to share common infrastructure while ensuring complete tenant isolation, customizable storefronts, secure commission-based payment processing, and simplified deployment, thereby reducing operational costs and improving accessibility for small businesses.

4. Root Cause Analysis

The increasing difficulty faced by small retailers in adopting digital commerce platforms is primarily caused by the high financial and technical requirements associated with conventional e-commerce systems. Existing solutions generally require separate infrastructure, individual software deployment, continuous maintenance, and dedicated operational resources for each retailer. These factors collectively increase implementation costs and reduce scalability.

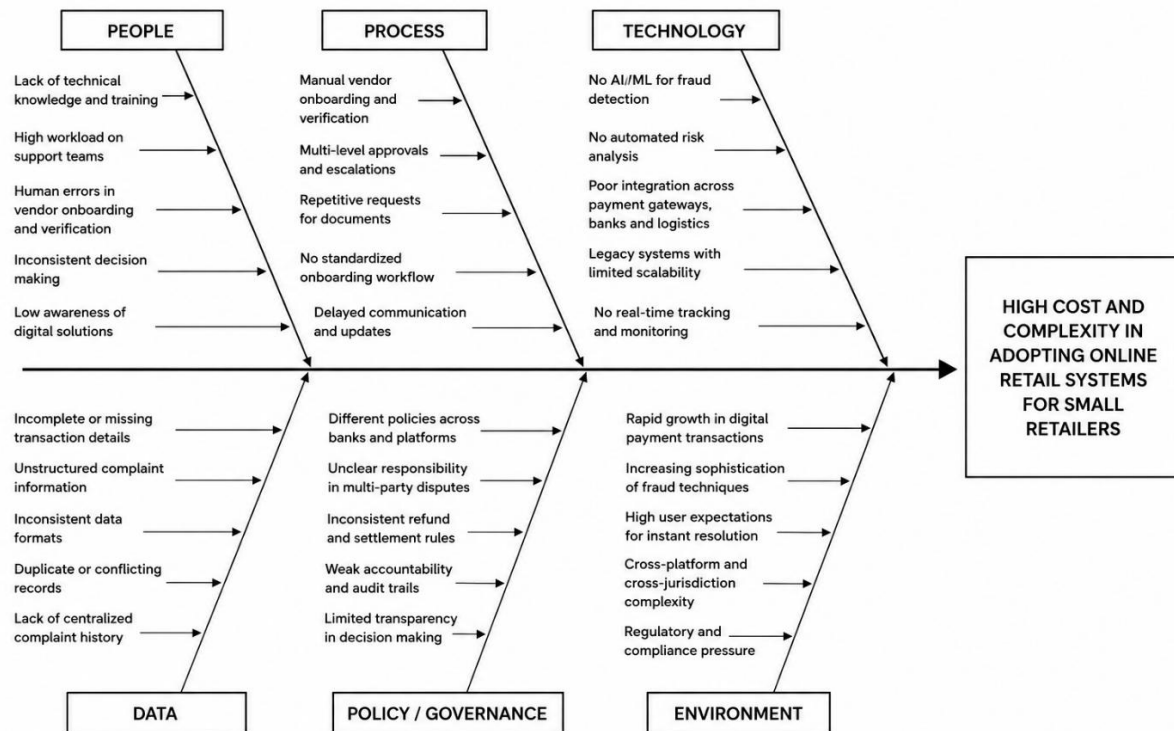
Furthermore, traditional systems often duplicate computing resources, require manual storefront customization, and provide limited flexibility for small businesses operating with constrained budgets. The absence of a shared multi-tenant architecture results in inefficient resource utilization and higher maintenance overhead. Consequently, many SMEs are unable to compete effectively in the growing online retail market.

The proposed Multi-Tenant SaaS framework addresses these root causes by introducing shared infrastructure, tenant-aware data isolation, reusable storefront templates, and a commission-based business model that minimizes upfront investment.

5 Why Analysis

Why Level	Analysis
Problem	Small retailers find it expensive to establish online stores.
Why 1?	Because existing e-commerce platforms require high upfront investment.
Why 2?	Because each retailer requires separate infrastructure, deployment, and maintenance.
Why 3?	Because traditional architectures do not effectively share computing resources among multiple vendors.
Why 4?	Because most systems follow single-tenant deployment models that duplicate hardware, software, and operational resources.
Why 5?	Because there is a lack of affordable, scalable, and secure multi-tenant SaaS architectures designed specifically for small online retailers.

Fishbone Diagram:



5. Systematic Literature Review:

Sr.No.	Title	Gaps	Findings
1	SaaS based Multi-tenant E-commerce to reduce cost and improve resource utilization	Focuses on general cost reduction; lacks specific domain mapping for B2B sub-tenants.	Multi-tenancy reduces development and maintenance costs by sharing codebases.
2	Multi-Tenant Architecture: A Comprehensive Framework for Building Scalable SaaS Applications	Theoretical framework; lacks specific implementation for commission-based retail models.	Defines isolation strategies (database vs. schema) for scalable SaaS.
3	A Multi-Vendor SaaS website builder with Role-Based CRM	Focuses on marketing agencies; does not detail the commission-split payment flow.	Discusses "all-in-one" platforms with integrated CRM and visual builders.
4	Multi-Tenant SaaS Architectures: Design Principles and Security Considerations	Heavy focus on security/compliance; less focus on frontend template customization	Compares single-tenant vs. multi-tenant database partitioning strategies.
5	Towards Dynamic Tenant Management for Microservice based Multi-Tenant SaaS Applications	Does not integrate storefront, domain resolution, or transaction commissions	Proposes a methodological framework to manage tenancy degree in microservice-based SaaS
6	Ensuring Highperformance of Mission-critical Java Applications in Multi-tenant Cloud Platforms	Concentrates on backend performance interference; does not address retail specific concerns such as storefront templating or commission settlement.	Identifies multi-tenant performance pathologies for Java workloads on shared platforms and emphasizes the need for isolation against adverse cross-tenant effects.

Sr.No.	Title	Gaps	Findings
7	Multi-Tenant Ecommerce based on SaaS Model to Minimize IT Cost	Limited attention to advanced isolation governance and generalized domain strategies.	Argues multi-tenant retail SaaS reduces IT cost for vendors and supports per-order/usage-contingent charging, reinforcing transactions-based affordability for small enterprises.
8	Cloud Based ECommerce Platform for Multi-Tenant and Multi-Vendor	Assumes a cloud stack; not focused on template storefront UX.	Proposes multi-tenant PaaS with REST APIs and cloud workflow; includes tenant tables and architecture diagrams.
9	A Systematic Mapping Study on the Multi-tenant Architecture of SaaS Systems	A survey paper; does not propose a specific architectural solution for retail.	Identifies customization and migration as major challenges in SaaS.
10	Thread-level resource consumption control of tenant custom code in a shared JVM for multi-tenant SaaS	Deep JVM-level solution space; does not address tenant facing retail concerns	Proposes controlling CPU/memory at the thread level for tenant code.
11	A Decision-based Model for Optimizing the Deployment of Cloud-hosted Application Components to Guarantee Multitenancy Isolation	Decision modeling does not materialize as a full retail SaaS stack; commission economics absent	Presents a model balancing degrees of component isolation against resource consumption and performance impact, supporting isolation-aware deployment decisions.
12	Transforming ECommerce Platforms through Multi-Tenant Architectures and Microservices	Storefront templating is not central.	Examines transforming retail platforms using multi-tenant architectures and microservices to address scalability and reliability demands.

6. Comparative Analysis

The reviewed literature indicates that existing research on multi-tenant SaaS systems primarily focuses on individual architectural aspects rather than providing a complete end-to-end retail platform. The major research directions identified are:

1. Multi-Tenant SaaS Cost Optimization

Several studies emphasize reducing infrastructure and maintenance costs by allowing multiple tenants to share a common application and database architecture. These approaches successfully improve resource utilization but provide limited support for advanced storefront customization and flexible monetization models.

2. Tenant Isolation and Security

A significant portion of the literature focuses on ensuring secure tenant isolation using shared-schema, shared-database, or isolated database architectures. These studies improve data privacy and security but generally overlook user experience, storefront personalization, and payment workflow integration.

3. Microservice-Based SaaS Architectures

Recent research proposes microservice-based architectures to improve scalability, deployment flexibility, and resource management. While these architectures enhance system performance,

they often lack integrated solutions for domain routing, commission-based payment settlement, and tenant-specific storefront generation.

4. Cloud-Based Multi-Tenant Platforms

Several cloud-native platforms demonstrate improved scalability and lower operational costs using cloud infrastructure and REST-based services. However, most implementations assume cloud expertise and provide limited support for dynamic storefront customization.

5. Performance and Resource Isolation

Research focusing on Java-based multi-tenant environments investigates resource isolation techniques to prevent performance degradation caused by resource-intensive tenants. These studies primarily address backend optimization rather than business functionality.

However, existing systems generally lack complete integration of:

- Shared-schema multi-tenant architecture with strict tenant isolation
- Commission-based payment split settlement
- Template-driven storefront generation
- Automatic subdomain and custom-domain routing
- Unified vendor onboarding workflow
- Cost-effective monetization model for SMEs
- Secure tenant-aware data access
- Scalable architecture supporting future microservice migration

SOTA Benchmarking Insight

Existing multi-tenant SaaS solutions primarily focus on scalability, cost reduction, and tenant security. However, they lack an integrated architecture for online retail. The proposed framework combines secure multi-tenancy, customizable storefronts, commission-based monetization, and domain-based tenant management into a unified, scalable, and affordable solution for small retailers.

7. Research Gap Identification

The literature review covers multi-tenant SaaS, database isolation, storefront builders, security, microservices, JVM performance, and cloud deployment. While previous studies show that multi-tenancy reduces infrastructure cost, improves scalability, and supports shared resources, they address these topics independently. No existing work provides a unified framework specifically designed for retail SaaS platforms.

Identified Research Gaps

- Existing studies focus on infrastructure sharing but do not integrate commission-based monetization, template-driven storefronts, and tenant management into a single retail architecture.
- Commission-based payment models are discussed conceptually but lack a standardized split-settlement mechanism within the system architecture.
- Database isolation research does not consider its integration with tenant-specific storefront customization and rendering.
- Most platforms support only subdomain-based tenant identification, while combined support for subdomains and custom domains is rarely explored.
- Performance isolation studies focus on generic cloud workloads rather than retail-specific shared-schema environments.
- Current literature provides limited quantitative comparison between multi-tenant commission-based platforms and traditional single-tenant retail solutions.

White Space Identification

Problem: Small online retailers require affordable, secure, and easily deployable e-commerce platforms.

Existing Solutions: Multi-tenant SaaS, database isolation, storefront builders, and tenancy management are available but are studied separately.

Research White Space: A unified shared-schema multi-tenant framework integrating commission-based settlement, template-driven storefronts, domain-based tenant resolution, and performance isolation for retail applications.

Justification

The proposed framework addresses a significant gap by combining cost efficiency, secure tenant isolation, flexible storefront customization, and reliable performance into a single architecture. This integrated approach is particularly beneficial for small retailers with limited budgets and can be evaluated against traditional single-tenant platforms using measurable performance and cost metrics.

8. Research Question

Primary Research Question

To what extent can a shared-schema multi-tenant SaaS architecture integrating commission-based settlement, template-driven storefront rendering, and domain-based tenant resolution reduce onboarding and operational costs for small online retailers while maintaining data isolation and acceptable performance compared with conventional single-tenant e-commerce platforms?

Measurable Sub-Questions

- **Cost Efficiency:** Does commission-based shared-schema multi-tenancy reduce onboarding and operational costs compared to licensed or dedicated platforms?
- **Isolation Integrity:** Can tenant-context enforcement prevent cross-tenant data access in a shared-schema environment?
- **Storefront Agility:** Does template-based storefront generation reduce deployment time and technical effort compared to custom development?
- **Routing Reliability:** Can the domain-resolution mechanism accurately support both subdomains and custom domains?
- **Performance Resilience:** Can resource management minimize noisy-neighbor performance issues under shared workloads?

Experimental Validation

The proposed framework will be evaluated through prototype implementation and comparison with a traditional single-tenant platform using measurable metrics:

- Cost comparison based on setup and recurring operational expenses.
- Automated testing to verify tenant data isolation.
- Deployment time comparison between template-based and custom storefront creation.
- Domain-routing accuracy and response-time measurement.
- Load-testing to evaluate performance stability under concurrent tenant traffic.

Alignment with Research Gap

The research directly addresses the identified gap by evaluating an integrated multi-tenant retail framework that combines cost optimization, secure tenant isolation, storefront customization, domain management, and performance governance. Unlike previous studies that examine these aspects separately, this work investigates their combined effectiveness within a unified architecture for small online retailers.

9. Hypothesis

Based on the identified research gap, the proposed multi-tenant SaaS framework is expected to outperform conventional single-tenant e-commerce platforms in terms of cost, deployment speed, tenant isolation, routing reliability, and overall scalability while maintaining acceptable performance under shared workloads.

Primary Hypothesis (H1)

A shared-schema multi-tenant SaaS framework integrating commission-based settlement, template-driven storefronts, and domain-based tenant resolution will significantly reduce per-tenant cost and deployment time while maintaining secure tenant isolation and acceptable system performance compared with a conventional single-tenant retail platform.

Quantifiable Hypotheses

- **H1a – Cost Reduction:** Reduce per-tenant onboarding and operational costs by at least **40%**.
- **H1b – Isolation Integrity:** Achieve **zero cross-tenant data leakage** through tenant-context enforcement.
- **H1c – Storefront Deployment:** Reduce storefront deployment time by at least **70%** using template-driven rendering.
- **H1d – Domain Resolution:** Achieve **99% or higher** tenant-routing accuracy with response latency below **50 ms**.
- **H1e – Settlement Accuracy:** Ensure at least **99.5%** accuracy in commission-based payment distribution.
- **H1f – Performance Isolation:** Limit response-time degradation under shared workloads to **less than 15%**.

Null Hypothesis (H0)

There is no significant difference between the proposed multi-tenant framework and a conventional single-tenant platform in terms of cost, deployment speed, tenant isolation, routing accuracy, settlement accuracy, or performance under shared load.

10. Baseline System Definition

To evaluate the proposed framework, a conventional **single-tenant/licensed e-commerce platform** is selected as the baseline because it represents the most common solution used by small online retailers.

Baseline System

In this model, each vendor purchases a licensed platform, subscribes to a fixed-fee hosted service, or develops a custom e-commerce website. Every vendor is provided with a dedicated application instance, database, or isolated environment.

Typical Workflow

- Vendor selects a licensed platform or custom development.
- Upfront costs are incurred for licensing, setup, and customization.
- A dedicated application and database are provisioned.
- Domain configuration is manually performed.
- Fixed maintenance or subscription fees are paid regardless of sales volume.

Baseline Characteristics

- Fixed upfront pricing model.
- Dedicated infrastructure for each vendor.
- Custom or theme-based storefront development.
- Manual domain configuration.
- Physical tenant isolation.
- Limited resource sharing and performance optimization.

Limitations

- **High Initial Cost:** Expensive licensing and development discourage small businesses.
- **Low Resource Utilization:** Dedicated infrastructure increases operational costs.
- **Slow Deployment:** Custom development requires more time than template-based solutions.
- **Manual Domain Management:** Domain setup is less flexible and requires manual configuration.
- **Fixed Costs:** Vendors incur expenses even when transaction volumes are low.

Justification

This baseline reflects the current approach adopted by many small and medium online retailers. It provides a realistic comparison for evaluating whether the proposed shared-schema, commission-based multi-tenant framework offers better affordability, scalability, and operational efficiency.

Purpose in This Study

The baseline serves as the reference system for evaluating the proposed framework. It enables direct comparison of cost, deployment time, tenant isolation, routing efficiency, and performance under identical workloads. Using a common baseline ensures that improvements measured through the defined KPIs are attributable to the proposed architecture rather than differences in workload or system configuration.

Comparison of Baseline System and Proposed Framework

Parameter	Baseline System	Proposed Multi-Tenant Framework
Tenant Provisioning	Manual setup of a dedicated instance/site per vendor	Automated onboarding onto shared schema + auto-assigned subdomain
Cost Model	Fixed upfront licensing / development fee, paid regardless of sales	Commission-only, transaction-contingent settlement
Storefront Creation	Custom front-end development or theme purchase per vendor	Template-injection rendering from persisted page schemas
Data Isolation	Physical isolation by dedicated database/instance	Logical isolation via enforced tenant-context scoping on shared schema
Domain Handling	Static, manually configured domain per instance	Dynamic subdomain + custom-domain resolution with caching
Resource Utilization	Under-utilized dedicated infrastructure per tenant	Shared, pooled infrastructure with tenant density optimization
Performance Governance	Not applicable (isolated by design)	Explicit noisy-neighbor mitigation and resource-level control

11. Evaluation Metrics (KPIs)

To move the hypotheses in Section 3 from qualitative claims to testable outcomes, each is attached to a specific, measurable Key Performance Indicator. The metrics below are chosen so that every architectural layer described in the proposed methodology — commission settlement, tenant isolation, storefront rendering, domain resolution, and performance governance — has at least one corresponding, independently measurable KPI. Together they allow the proposed framework and the baseline system to be scored on a common, comparable basis rather than through anecdotal or purely descriptive comparison.

Metric	Description	Unit
Per-Tenant Cost Reduction	Reduction in upfront + operational cost per vendor versus a dedicated/licensed platform of equivalent capability	% reduction

Metric	Description	Unit
Tenant Isolation Integrity	Rate of cross-tenant data-leakage incidents observed during scoped-query and load testing across simulated tenants	Incidents / 10,000 requests
Storefront Deployment Time	Time for a tenant to launch a functional storefront using template injection versus custom front-end development	Hours
Domain Resolution Accuracy	Percentage of requests (subdomain and custom-domain) correctly mapped to the owning tenant ID	%
Domain Resolution Latency	Time taken to resolve an incoming Host header to a tenant ID, including cached custom-domain lookups	ms
Commission Settlement Accuracy	Percentage of split-payment transactions correctly apportioned between tenant and platform at the configured commission rate	%
Tenant Density	Number of active tenants supported per compute/database instance under the shared-schema model	Tenants / instance
Performance Degradation (Noisy-Neighbor)	Increase in response time for a tenant under contention from co-located high-load tenants	%

12. Proposed Methodology

The proposed methodology operationalizes the framework described in the source paper as a layered system in which vendor-facing storefronts, tenant isolation, domain resolution, and commission settlement are treated as first-class architectural concerns rather than incidental implementation details. The design concentrates engineering effort on a single shared codebase and operating substrate, consistent with multi-tenant SaaS cost-reduction arguments,

Layer 1 — Domain Resolution

Every incoming request is first inspected at the Host header. If the host matches the platform's canonical subdomain pattern, the tenant key is extracted directly. Otherwise, the host is checked against a cached domain-alias registry to resolve custom-domain tenants. This layer determines which tenant a request belongs to before any business logic executes.

Layer 2 — Tenant Context Middleware

The resolved tenant ID is bound to the request context and propagated through every subsequent layer. Tenant scoping is enforced centrally in the data-access layer rather than left to per-query discipline, forming a consistent barrier against cross-tenant access at every system entry point.

Layer 3 — Application Layer (Template-Injection Engine and Business Logic)

Two cooperating modules operate under the tenant context: the template-injection rendering engine, which builds storefronts from tenant-selected, persisted page schemas rather than custom code, and the business-logic/API layer, which handles onboarding, catalog and cart operations, and order orchestration.

Layer 4 — Shared-Schema Data Layer

Tenant data is stored in shared tables with a tenant discriminator that scopes every operation. Centralized scoping in the data-access layer, optionally reinforced with row-level policies at the database boundary, mitigates the principal risk of shared-schema designs — a mis-scoped query exposing one tenant's data to another.

Layer 5 — Commission Settlement Engine

At payment capture, the platform issues a split-payment request naming both the tenant and the provider as recipients. The provider's share ($\text{commission} \times \text{amount}$) and the tenant's share ($(1 - \text{commission}) \times \text{amount}$) are computed against a configurable commission band, and the split is confirmed via gateway webhooks before the order status is synchronously updated.

Layer 6 — Performance Isolation and Monitoring

Because the reference implementation targets a Java-centric shared runtime, this layer applies fine-grained (e.g., thread-level) resource control and runtime observability to contain noisy-neighbor interference, ensuring that load from one tenant does not degrade service for co-located tenants.

System Architecture

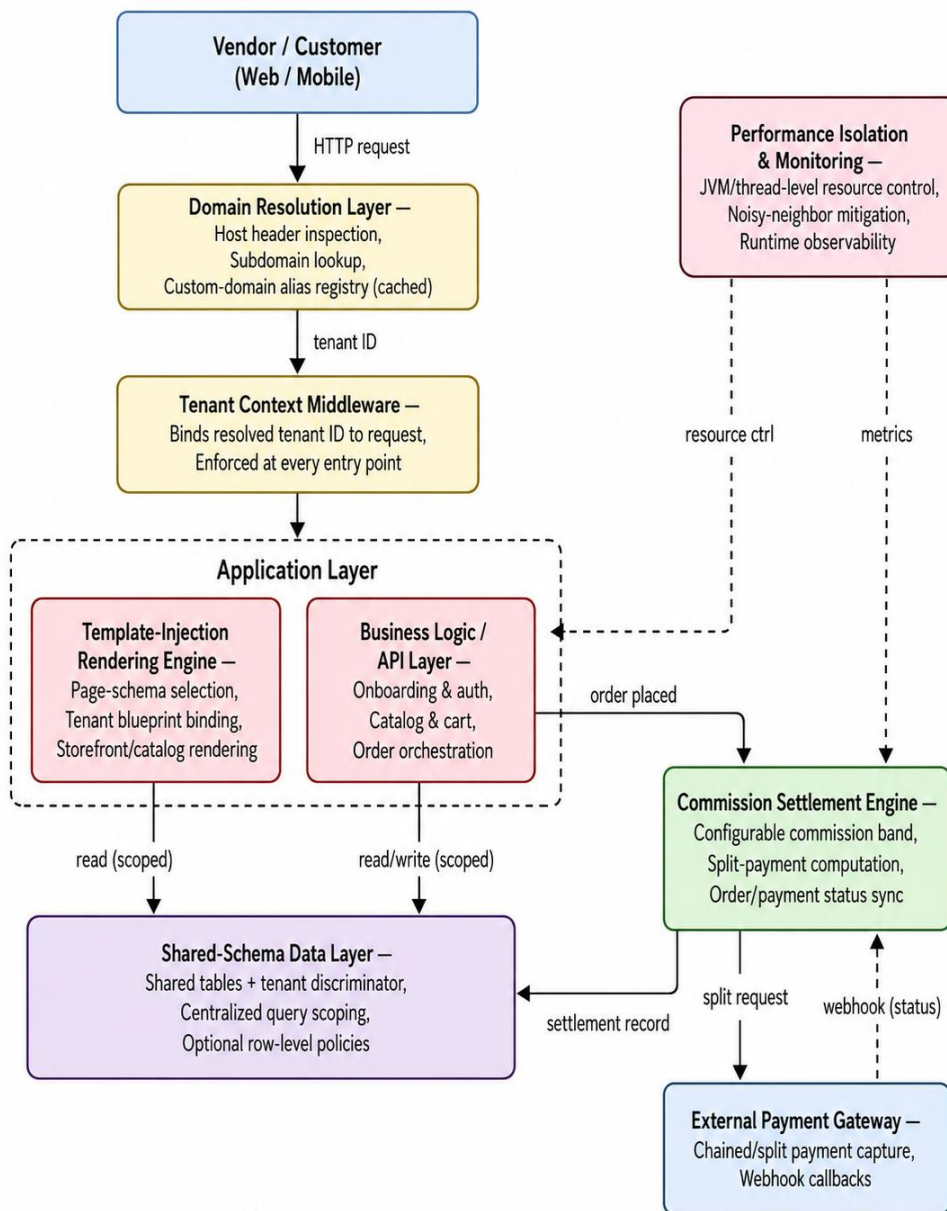


Figure 1: System Architecture of the Proposed Multi-Tenant SaaS Framework

This layered structure ensures that cost efficiency, isolation, storefront agility, domain-resolution reliability, and performance stability are each addressed by a dedicated architectural component, directly aligned with the KPIs defined in Section 5 and testable against the baseline defined in Section 4.

13. Feasibility Analysis

Model: TELOS Framework

Technical Feasibility

The proposed framework is technically feasible because each of its layers builds on demonstrated, currently available technology. Domain-based tenant resolution using Host-header inspection and a cached domain-alias registry is a well-established pattern in subdomain-routed SaaS publishing systems. The shared-schema data layer, with centrally enforced tenant-context scoping and optional row-level policies at the database boundary, is achievable using standard relational database features. The template-injection rendering engine requires only a persisted page-schema store and a rendering pipeline that plugs tenant content into a small library of pre-defined designs (typically 4-5), avoiding custom front-end development per tenant. Commission-based settlement can be implemented using existing payment-gateway split-payment APIs that support multi-recipient transfers confirmed through webhook callbacks. Because the reference implementation targets a Java-centric runtime, performance isolation is technically supported through fine-grained, thread-level resource control mechanisms already documented for shared multi-tenant JVM environments.

Economic Feasibility

The framework's monetization model is itself an economic-feasibility argument: because platform revenue is realized only as a configurable percentage of tenant transactions, provider income scales with adoption rather than depending on upfront capital recovery. A shared codebase and pooled infrastructure reduce the provider's per-tenant hosting and maintenance cost relative to provisioning a dedicated instance per vendor, while tenants avoid licensing, customization, and infrastructure costs altogether and pay only once they earn revenue.

Legal Feasibility

Commission settlement is routed through the payment gateway's regulated split-payment mechanism rather than the platform directly holding tenant funds, keeping fund movement within an already-regulated payment-processing channel. Tenant data isolation enforced at the application and data-access layers supports the confidentiality expectations that arise from hosting multiple independent vendors' commercial data on shared infrastructure.

Operational Feasibility

The system's operational demand on vendors is deliberately minimized: onboarding automatically provisions a tenant ID and domain, storefront creation is template-based rather than code-based, and logistics and order fulfilment remain the vendor's own responsibility rather than a platform obligation. This keeps the platform's operational scope centered on identity, storefront rendering, transaction processing, and tenant isolation, consistent with the programmable-core positioning described in the methodology.

Schedule Feasibility

Because the architecture is explicitly layered - domain resolution, tenant-context middleware, the application/template-injection layer, the shared-schema data layer, the commission settlement engine, and the performance isolation layer - it lends itself to phased, modular implementation. Core tenant resolution and shared-schema isolation can be built first, followed by the template-injection storefront engine, then the commission settlement engine, with performance-isolation controls added as tenant density increases.

14. Stakeholder Analysis

Model: Tri-Level Stakeholder Model

Primary Stakeholders

- Small and medium-sized online retailers (tenants) who onboard onto the platform to obtain a storefront without upfront investment.
- The platform provider, whose revenue is directly tied to tenant transaction volume under the commission-only monetization model.

Secondary Stakeholders

- End customers/shoppers who browse and purchase through tenant storefronts, whose experience depends on correct domain resolution and consistent template rendering.
- Payment gateway/service providers, who execute the split-settlement transaction between tenant and platform provider at the moment of payment capture.

Tertiary Stakeholders

- Future developers of AI-driven storefront-generation agents and analytics/forecasting modules, identified in the framework's future scope as natural extensions once the platform accumulates page-schema and event-stream data.
- The broader research community working on cloud computing, multi-tenancy, isolation, and performance engineering, whose foundational work underpins the architectural choices made in this framework.

This layered structure ensures that every participant affected by tenant onboarding, storefront rendering, payment settlement, and platform evolution is accounted for in the research design.

15. Expected Contribution and Novelty

1. New Method / Architecture

A hybrid architecture combining:

- Shared-schema multi-tenancy with cross-layer tenant-context enforcement
- A template-injection front-end rendering engine driven by persisted page schemas rather than custom code
- Dual-mode domain resolution supporting both subdomain and custom-domain tenant identification with cached lookups
- Commission-based split-payment settlement at the point of payment capture
- Performance isolation for a Java-centric shared runtime through fine-grained, thread-level resource control

2. Performance Improvement

- Reduction in per-tenant onboarding and operational cost relative to a dedicated/licensed baseline platform
- Reduction in storefront deployment time through template-driven rendering instead of custom development
- High-accuracy, low-latency domain resolution across both subdomain and custom-domain routing paths
- Bounded performance degradation for co-located tenants under contention (noisy-neighbor mitigation)

3. Dataset / Benchmark Contribution

- A KPI-based benchmarking protocol - cost reduction, isolation integrity, deployment time, routing accuracy/latency, settlement accuracy, tenant density, and performance degradation - that allows the proposed framework to be scored against the baseline system on a common, measurable basis rather than through descriptive comparison alone.

4. Application Innovation

First integrated retail-focused SaaS platform to combine commission-only monetization, template-driven storefront generation, dual-mode domain resolution, and enforced shared-schema isolation within a single architecture, rather than addressing these concerns as separate, disconnected systems as observed in the reviewed literature.

Novelty is:

Non-trivial, Defensible (a clear, literature-supported gap, established in Section 7) and Research-worthy

16. Research Design Summary

Research Question:

To what extent can a shared-schema multi-tenant SaaS architecture integrating commission-based settlement, template-driven storefront rendering, and domain-based tenant resolution reduce onboarding and operational costs for small online retailers while maintaining data isolation and acceptable performance compared with conventional single-tenant e-commerce platforms?

Hypothesis:

A shared-schema multi-tenant SaaS framework integrating commission-based settlement, template-driven storefronts, and domain-based tenant resolution will significantly reduce per-tenant cost and deployment time while maintaining secure tenant isolation and acceptable system performance compared with a conventional single-tenant retail platform.

Baseline:

A conventional single-tenant/licensed e-commerce platform in which each vendor purchases or develops a dedicated application instance, database, and domain configuration under a fixed upfront pricing model.

Metrics:

- Per-Tenant Cost Reduction
- Tenant Isolation Integrity
- Storefront Deployment Time
- Domain Resolution Accuracy and Latency
- Commission Settlement Accuracy
- Tenant Density
- Performance Degradation (Noisy-Neighbor)

Contribution:

A novel shared-schema multi-tenant SaaS framework that integrates commission-based settlement, template-injection storefront rendering, dual-mode domain resolution, and Java-centric performance isolation into a single, cost-aligned architecture for small online retailers.

Declaration

This work represents original and evidence-based research preparation.

Appendix A: Research Quality Standards

Relevance:

The research addresses a practical and widespread barrier facing small and medium online retailers: the fixed upfront cost structure of conventional e-commerce platforms. As digital commerce and small-vendor participation continue to grow, an affordable, transaction-contingent alternative is directly relevant to the current state of the online retail ecosystem.

Rigor:

The study follows a structured methodology, using accepted analytical models - 5-Why Analysis, the Fishbone (Ishikawa) Model, SOTA Benchmarking, White-Space Identification, and the TELOS Feasibility Framework - to move from problem identification through to a testable, layered architectural proposal. Each stage of the research, from root-cause analysis to KPI definition, is grounded in the reviewed literature rather than assumption.

Reproducibility:

The framework defines clearly bounded inputs (tenant onboarding data, page schemas, transaction events), processing stages (domain resolution, tenant-context binding, template rendering, commission split), and measurable KPIs (cost reduction, isolation integrity, deployment time, routing accuracy/latency, settlement accuracy, tenant density, performance degradation). This allows the same methodology to be replicated and benchmarked under comparable conditions by future researchers.

Responsibility:

The research treats tenant data isolation as a first-class architectural concern rather than an afterthought, enforcing tenant-context scoping at every system entry point and reinforcing it, where needed, with database-level policies. Commission settlement is routed through the payment gateway's regulated split-payment mechanism rather than platform-held funds, and the platform's scope is deliberately bounded to identity, storefront rendering, and transaction processing, leaving logistics and fulfilment as the vendor's responsibility.

Result Orientation:

The study is outcome-focused, defining measurable targets for cost reduction, isolation integrity, deployment speed, routing accuracy, settlement accuracy, and performance stability, so that the proposed framework's value can be demonstrated against the baseline system rather than argued only in qualitative terms.

Appendix B: Evaluation Framework

Problem Clarity:

The research clearly identifies the specific cost and architectural barriers preventing small retailers from adopting digital commerce: fixed upfront licensing and development costs, dedicated per-vendor infrastructure, manual storefront customization, and manual domain configuration.

Gap Identification:

Through the systematic literature review and comparative analysis, the research shows that existing work addresses multi-tenancy cost reduction, tenant isolation, storefront builders, microservice architectures, and Java-based performance isolation as separate concerns. No reviewed work integrates commission-based settlement, template-driven storefronts, and dual-mode domain resolution into a single retail-focused framework, which justifies the proposed architecture.

Research Question + Hypothesis:

The study formulates a measurable research question on whether the proposed shared-schema, commission-based multi-tenant framework can outperform a conventional single-tenant baseline on cost, isolation, deployment speed, routing reliability, and performance, and pairs it with a quantified hypothesis (H1a-H1f) rather than a purely qualitative claim.

Methodology Logic:

The methodology proceeds through a logical, layered sequence: domain resolution, tenant-context middleware, the application/template-injection layer, the shared-schema data layer, the commission settlement engine, and performance isolation. Each layer is mapped to a specific weakness identified in the baseline system, giving the design internal coherence.

Novelty:

The proposed framework introduces innovative elements - enforced tenant-context scoping across every architectural layer, a constrained template-injection rendering engine, dual-mode (subdomain and custom-domain) resolution with caching, and configurable commission-band split settlement - combined into a single retail-specific architecture, which distinguishes it from the isolated contributions found in the reviewed literature.

Appendix C: Common Failure Modes

Solution-First Thinking: A common failure in student research is starting from a technology (for example, "build it in Java" or "use microservices") without first establishing the underlying problem. This study avoids that by beginning with the cost barrier facing small retailers before proposing multi-tenancy, template injection, and commission settlement as responses to that specific problem.

Weak or Missing Gap: Research that does not clearly state what is missing from existing systems risks appearing repetitive. This study addresses that risk directly in Section 7 by naming the specific combination - commission-based settlement, template-driven storefronts, and dual-domain resolution within one shared-schema architecture - that the reviewed literature does not provide together.

No Measurable Metrics: Claims of "lower cost" or "better performance" without defined measurement are of limited academic value. This study defines specific, unit-bound KPIs in Section 11 - percentage cost reduction, incidents per 10,000 requests, hours to deploy, percentage routing accuracy, millisecond latency, percentage settlement accuracy, tenants per instance, and percentage performance degradation - so every claim in the hypothesis is independently testable.

Copy-Based Work: Reproducing an existing multi-tenant SaaS platform with only cosmetic changes would not constitute a research contribution. This study avoids that by combining shared-schema isolation, template injection, dual-mode domain resolution, and commission-based settlement into one architecture, a combination not found as a single system in the reviewed literature.

Appendix D: Model Justification, Mathematical Formulation, and Usage

This appendix provides formal justification, mathematical representation, and application guidelines for the analytical and research models used in this study. Each model was selected based on its relevance to the problem structure, its analytical depth, and its ability to support evidence-based conclusions.

D1. Root Cause Analysis Models

1. 5-Why Analysis

Justification: Used to trace the underlying causes of the high-cost barrier facing small online retailers back to a single root cause rather than stopping at the visible symptom.

Formulation: $P_0 \rightarrow C_1 \rightarrow C_2 \rightarrow \dots \rightarrow C_n$, where C_n represents the root cause.

Usage in this Study:

- Identify why existing e-commerce platforms require high upfront investment
- Trace this to per-vendor dedicated infrastructure, deployment, and maintenance
- Trace this further to the absence of shared, resource-pooled architectures
- Validate the root cause - lack of an affordable, secure, multi-tenant SaaS architecture - before proposing the framework

2. Fishbone (Ishikawa) Model

Justification: Provides structured categorization of the multiple contributing factors behind high vendor-onboarding cost and inefficiency.

Formulation: $\text{TotalCauses} = \sum_{i=1}^k C_i$.

Usage in this Study:

- Categorize causes affecting cost and deployment efficiency across relevant factor groups
- Identify technical barriers such as the lack of shared-schema isolation and manual domain configuration
- Prioritize the major contributors driving the fixed-cost adoption barrier

D2. Literature Review Model

SOTA Benchmarking

Justification: Enables direct comparison with existing multi-tenant SaaS, storefront-builder, and Java performance-isolation systems.

Formulation: $\text{PerformanceGap} = P_{\text{proposed}} - P_{\text{SOTA}}$.

Usage in this Study:

- Compare the proposed cost model with existing licensed/dedicated e-commerce platforms
- Benchmark storefront customization approaches against existing template/builder systems
- Identify the absence of dual-mode domain resolution and commission-based settlement in current platforms
- Establish the reference baseline used in Section 10

D3. Research Gap Identification Model**White-Space Identification**

Justification: Identifies unexplored territory within the current multi-tenant SaaS and e-commerce literature.

Formulation: Gap = ProblemSpace minus ExploredSolutions.

Usage in this Study:

- Map existing multi-tenant, isolation, storefront-builder, and performance-isolation solutions
- Identify the absence of an integrated commission-settlement + template-injection + dual-domain-resolution framework
- Define the specific research opportunity addressed in Section 7

D4. Feasibility Analysis Model**TELOS Framework**

Justification: Validates system feasibility across multiple independent dimensions rather than technical feasibility alone.

Formulation: $F = f(T, E, L, O, S)$.

Usage in this Study:

- Assess the technical feasibility of the shared-schema, template-injection, and commission-settlement layers
- Evaluate the economic feasibility of the commission-only monetization model
- Consider the legal feasibility of gateway-routed split payments and tenant data isolation
- Analyze the operational feasibility of vendor onboarding and template-based storefront creation
- Support phased, layer-by-layer implementation planning, as set out in Section 13

D5. Stakeholder Analysis Model

Tri-Level Stakeholder Model

Justification: Ensures all parties affected by or influencing the framework are systematically identified across three tiers of proximity to the system.

Formulation: $S = \{ S \text{ primary}, S \text{ secondary}, S \text{ tertiary} \}$.

Usage in this Study:

- Classify tenants and the platform provider as primary stakeholders
- Classify end customers and payment gateway providers as secondary stakeholders
- Classify future analytics/AI-agent developers and the research community as tertiary stakeholders

D6. Evaluation Metrics Model

General Performance Function

Justification: Quantifies improvement of the proposed framework over the baseline system in measurable percentage terms.

Formulation: $\text{Improvement}(\%) = ((P \text{ new} - P \text{ baseline}) / P \text{ baseline}) \times 100$.

Usage in this Study:

- Measure the percentage reduction in per-tenant onboarding and operational cost
- Measure the percentage reduction in storefront deployment time
- Compare domain-resolution accuracy and settlement accuracy against baseline levels
- Validate the quantified hypotheses (H1a-H1f) defined in Section 9 using measurable evidence

D7. Model Selection Justification Summary

The models selected for this study support distinct stages of the research process: root-cause models (5-Why, Fishbone) identify the fundamental cause of the cost barrier; the White-Space model identifies the specific unexplored combination that justifies the framework; the TELOS model validates feasibility across technical, economic, legal, operational, and schedule dimensions; the Tri-Level Stakeholder Model ensures complete stakeholder coverage; and the General Performance Function ties every KPI in Section 11 back to a measurable, baseline-relative improvement. Together, these models give the research analytical rigor, logical consistency across stages, and reproducibility.